



OASIS Security Report

XML External Entity Injection Security Analysis

Date: 2026-06-12 01:38:32 — **Model:** llama3.2:3b

Vulnerability: XML External Entity Injection

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **4** file(s) for XML External Entity Injection.

[Return to table of contents](#)

Statistics

Files analyzed: 4 **Total findings:** 5

Critical



4

High		0
Medium		1
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.644
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.623
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>	0.605
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>	0.586

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.644

Source lines: 1-39

► Finding 1.1: Vulnerable XML parsing (Medium)

Explanation

The contract uses a DTD without proper entity restrictions, making it vulnerable to XML External Entity Injection.

Impact

Potential SSRF or file disclosure attacks

Entry point

/withdrawFunds

HTTP methods

POST

Parameters

_weiToWithdraw

Exploitation steps

- Attacker sends a malicious XML payload as the _weiToWithdraw value

Conditions

The contract uses a DTD without proper entity restrictions.

Remediation

Disable DTDs and external entities in XML parsers

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.623

Source lines: 1-85

► Finding 2.1: Vulnerability found (Critical)

Vulnerable code

```
<% Log.AddMessage(msg.sender, _am, "CashOut"); %>
```

Explanation

The Log contract uses a script to execute the AddMessage function with user-provided input (_am) without proper validation or sanitization.

Impact

Remote code execution and potential data theft

Entry point

/CashOut

Execution path

```
AddMessage(msg.sender, _am, "CashOut")  
Call contract at /CashOut  
Execute function with user-provided input
```

Parameters

_am

Exploitation steps

- Attackers manipulate the value of _am to execute arbitrary code
- Send a maliciously crafted _am value to trigger the attack

Example payloads

- ```
<% Log.AddMessage(msg.sender, 0x80, "CashOut"); %>
```

#### Conditions

The caller has sufficient authorization and the contract is in a vulnerable state

#### Remediation

Properly validate and sanitize user input, such as using require or assert statements to enforce input constraints.

## Secure example

```
% Log.AddMessage(msg.sender, _am, "Deposit");
```

## File 3: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.605

Source lines: 126-467

### ► Finding 3.1: XML External Entity Injection (Critical)

#### Vulnerable code

```
<![CDATA[<xslt xmlns='http://www.w3.org/1999/XSL/Transform'> <xsl:value-of
select='..' /></xslt>]]>
```

#### Explanation

The `_walletLibrary.delegatecall(msg.data)` call does not validate the input data, allowing an attacker to inject XML External Entities.

#### Impact

File disclosure, SSRF, denial of service, or data theft

#### Entry point

`function hasConfirmed(bytes32 _operation, address _owner)`

#### Execution path

```
delegatecall(msg.data) -> delegatecall(_walletLibrary.delegatecall(msg.data))
-> _walletLibrary.delegatecall()
```

#### Parameters

`msg.data`

#### Exploitation steps

- Attackers inject an XML External Entity into `msg.data`
- `Delegatecall` triggers the injection

#### Example payloads

- `<xslt xmlns='http://www.w3.org/1999/XSL/Transform'> <xsl:value-of  
select='..' /></xslt>`

#### Conditions

`msg.sender` is authenticated and has access to `delegatecall`

#### Remediation

Validate and sanitize input data, including `msg.data`

## Secure example

```
function hasConfirmed(bytes32 _operation, address _owner) external constant
returns (bool) { if (_walletLibrary.delegatecall(msg.data) == 0x0) { return
true; } else { return false; }}
```

## File 4: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.586

Source lines: 1-286

### ► Finding 4.1: Vulnerability found (Critical)

#### Vulnerable code

```
function revoke(bytes32 _operation) external {
 uint ownerIndex = m_ownerIndex[uint(msg.sender)];
 // make sure they're an owner
 if (ownerIndex == 0) return;
 uint ownerIndexBit = 2**ownerIndex;
 var pending = m_pending[_operation];
 if (pending.ownersDone & ownerIndexBit > 0) {
 pending.yetNeeded++;
 pending.ownersDone -= ownerIndexBit;
 Revoke(msg.sender, _operation);
 }
}
```

#### Explanation

This function allows an attacker to inject external entities by manipulating the `\_operation` parameter.

#### Impact

Potential for data theft or denial of service

#### Entry point

/revoke

#### Execution path

```
getOwner(uint ownerIndex) -> revoke(bytes32 _operation) -> Revoke(msg.sender,
_operation)
execute(address _to, uint _value, bytes _data) -> confirm(bytes32 o_hash) ->
revoke(o_hash)
```

#### Parameters

\_operation

#### Exploitation steps

- Manipulate the `\_operation` parameter to inject external entities.
- Send a request with an injected entity (e.g. `` ) to the `/revoke` endpoint.



## Example payloads

- `<x>`
- `<y>`

## Conditions

Access to the `/revoke` endpoint while authenticated.

## Remediation

Validate and sanitize the `\_operation` parameter to prevent external entity injection.

## Secure example

```
function revoke(bytes32 _operation) external {
 // Validate and sanitize the operation parameter
 if (!isValidOperation(_operation)) return;
 uint ownerIndex = m_ownerIndex[uint(msg.sender)];
 // ...
}
```

Source lines: 96–414

### ► Finding 4.1: Vulnerability found (Critical)

#### Vulnerable code

```
<![CDATA[exp]]>
```

#### Explanation

The `m\_pending[\_operation]` object is not properly sanitized, allowing for XML External Entity Injection attacks.

#### Impact

Potential data theft or file disclosure

#### Entry point

confirmAndCheck(bytes32 \_operation) internal returns (bool)

#### Execution path

```
confirmAndCheck -> m_pending[_operation] -> <![CDATA[exp]]>
```

#### Parameters

\_operation

#### Exploitation steps

- Exploit confirmAndCheck with malformed input
- Allow XML External Entity Injection via m\_pending[\_operation]

**Example payloads**

- `<!DOCTYPE html><html><body>Hello World!</body></html>`

**Conditions**

Unvalidated user input for \_operation

**Remediation**

Sanitize and validate user input for \_operation in confirmAndCheck

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)